

COMPANION
to

ChatGPT Teaches TikZ

Chapter 28: Building Complex Diagrams (A)

Contents

28.1	What This Companion Adds	2
28.2	An Architecture Vocabulary	3
28.3	Make a Structural Inventory	4
28.4	Define the Visual Contract First	4
28.5	Order the Source by Dependencies	5
28.6	Test the Skeleton Before Decoration	6
28.7	Keep Exceptional Geometry Explicit	6
28.8	Calculate Supporting Geometry	7
28.9	Regions Must Follow Their Members	7
28.10	Worked Project: A Comparison Diagram	7
28.11	A Revision Test Matrix	9
28.12	Debugging Workshop	10
28.13	Exercises	11
28.14	Solutions	11
28.15	ChatGPT Workshop	13
28.16	Final Checklist	13

28.1 What This Companion Adds

Chapter 28 assembled familiar techniques into substantial diagrams. This Companion supplies an architectural method: inventory the mathematical roles, separate dependencies from decoration, budget exceptions, and test the figure with deliberate revisions before polishing it.

Design

Complexity belongs in the mathematical relationships, not in scattered local corrections. Give each part of the source one recognizable responsibility.

28.2 An Architecture Vocabulary

Source element	Responsibility
Semantic style	Defines the appearance of one mathematical role.
Named node	Represents one important mathematical object.
Named coordinate	Records a reusable geometric dependency.
Relative placement	Expresses alignment and neighborhood.
Loop or pic	Encodes genuine repeated structure.
Explicit exception	Preserves a unique object without false abstraction.
Fitted node	Makes a region follow its members.
Background layer	Keeps supporting graphics behind the content.
Revision test	Exposes hidden dependence on current labels or spacing.

28.3 Make a Structural Inventory

Before writing TikZ commands, list the kinds of objects and relationships in the figure. A two-row mathematical diagram might contain ordinary objects, two key objects, horizontal maps, vertical maps, one exceptional comparison map, and two conceptual regions.

That inventory suggests source names:

```
object
key object
map
comparison
map label
region
```

Names based on roles remain meaningful after a design change. Names such as **blue box** and **thick arrow** merely repeat the present styling.

28.4 Define the Visual Contract First

```
\tikzset{
  object/.style={draw,rounded corners,
    minimum width=13mm,minimum height=8mm},
  key object/.style={object,very thick},
  map/.style={-{Stealth},thick},
  comparison/.style={map,dashed},
  map label/.style={fill=white,inner sep=1pt},
  region/.style={draw,rounded corners,inner sep=5mm}
}
```

The figure body can now say what each element is. The style block remains the single place where a visual convention is changed.

Common Mistake

Do not create a style for every unique object. A style is valuable when its name represents a recurring role or a project-wide decision.

28.5 Order the Source by Dependencies

Place an object before commands that depend on its anchors. Compute a coordinate after its inputs exist. Draw a fitted region after its member nodes exist, even if the region will be painted behind them.

```
% 1. principal nodes
\node[object] (A) {$A$};
\node[object,right=of A] (B) {$B$};

% 2. dependent geometry and edges
\coordinate (M) at ($(A)!0.5!(B)$);
\draw[map] (A)--(B);

% 3. supporting region, painted in back
\begin{scope}[on background layer]
  \node[region,fit=(A)(B)] {};
\end{scope}
```

Source order should follow logical availability; layers control painting order independently.

28.6 Test the Skeleton Before Decoration

The first compilable version should contain only principal nodes and structural edges.

```
\node[object] (A) {$A$};
\node[object,right=of A] (B) {$B$};
\node[object,below=of A] (C) {$C$};
\node[object,right=of C] (D) {$D$};
\draw[map] (A)--(B) (A)--(C)
           (B)--(D) (C)--(D);
```

If the skeleton is wrong, labels and backgrounds only make the error harder to see. Add secondary information after the structural arrangement is correct.

28.7 Keep Exceptional Geometry Explicit

A loop is appropriate for a uniform family. It is not a goal in itself. A single diagonal comparison map can remain explicit:

```
\draw[comparison]
  (A.south east) to[bend right=18]
  node[map label,below left] {$h$}
  (D.north west);
```

This source is easier to understand than a complicated data table designed to hide one exception.

Design

Abstract repetitions; name dependencies; leave genuine exceptions visible. This balance keeps the source compact without making it cryptic.

28.8 Calculate Supporting Geometry

Derived labels and annotations should follow the objects they describe.

```
\coordinate (row midpoint) at ($(A)!0.5!(B)$);  
\node[above=4mm of row midpoint] {First stage};
```

Use named coordinates for important dependencies and inline calculations for simple one-time relationships. Too many trivial names can be as distracting as too many raw coordinates.

28.9 Regions Must Follow Their Members

```
\begin{scope}[on background layer]  
  \node[region,fit=(A)(B)(C)(D)] (core) {};  
\end{scope}  
\node[above] at (core.north) {Core};
```

The fitted node follows changes in labels, node sizes, and relative positions. The background layer prevents it from covering the mathematical content.

28.10 Worked Project: A Comparison Diagram

28.10.1 Stage 1: Place the Objects

```
\node[object] (A) {$A$};  
\node[key object,right=of A] (B) {$B$};  
\node[object,right=of B] (C) {$C$};  
\node[object,below=of A] (D) {$D$};  
\node[key object,below=of B] (E) {$E$};  
\node[object,below=of C] (F) {$F$};
```

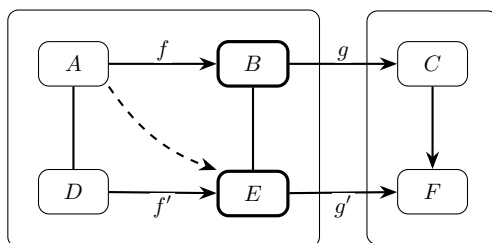
28.10.2 Stage 2: Add the Structural Maps

```
\draw[map] (A)--node[map label,above] {$f$} (B);  
\draw[map] (B)--node[map label,above] {$g$} (C);  
\draw[map] (D)--node[map label,below] {$f'$} (E);  
\draw[map] (E)--node[map label,below] {$g'$} (F);  
\draw[map] (A)--(D) (B)--(E) (C)--(F);
```

28.10.3 The Complete Picture

```
\begin{tikzpicture}[node distance=14mm and 19mm]
  \node[object] (A) {$A$};
  \node[key object,right=of A] (B) {$B$};
  \node[object,right=of B] (C) {$C$};
  \node[object,below=of A] (D) {$D$};
  \node[key object,below=of B] (E) {$E$};
  \node[object,below=of C] (F) {$F$};
  \draw[map] (A)--node[map label,above] {$f$} (B);
  \draw[map] (B)--node[map label,above] {$g$} (C);
  \draw[map] (D)--node[map label,below] {$f'$} (E);
  \draw[map] (E)--node[map label,below] {$g'$} (F);
  \draw[map] (A)--(D) (B)--(E) (C)--(F);
  \draw[comparison] (A.south east)
    to[bend right=18] (E.north west);
  \begin{scope}[on background layer]
    \node[region,fit=(A)(B)(D)(E)] {};
    \node[region,fit=(C)(F)] {};
  \end{scope}
\end{tikzpicture}
```

See



28.11 A Revision Test Matrix

Test one kind of dependency at a time.

Revision	What it tests
Lengthen a node label	Relative spacing, anchors, and fitted regions.
Change node dimensions	Edge attachment and region padding.
Move one row	Calculated points and orthogonal routes.
Restyle all maps	Centralization of the visual vocabulary.
Insert one object	Naming, source order, and local dependencies.

28.12 Debugging Workshop

28.12.1 A Style Change Misses One Object

That object probably contains duplicated local options instead of its semantic style.

28.12.2 A Region Stays Behind

Fit it to named objects rather than drawing a measured rectangle.

28.12.3 A Loop Contains Many Special Cases

Reduce the loop to the uniform family and write exceptional objects explicitly.

28.12.4 The Second Version Requires Many Repairs

Identify which numerical positions should become relative placements, anchors, or calculated coordinates.

28.13 Exercises

1. Inventory the roles in a six-object mathematical diagram.
2. Define semantic styles for its objects, maps, and labels.
3. Build only the structural skeleton.
4. Add one explicit exceptional connection.
5. Add a fitted background region.
6. Replace one measured point by a calculated coordinate.
7. Lengthen a label and record what changes automatically.
8. Change one style and verify that every matching object updates.

28.14 Solutions

28.14.1 Exercises 1–4

List mathematical roles before visual options. Define one style per recurring role, name each principal object, and connect the skeleton before adding labels. Keep the exceptional connection as its own `\draw` command.

28.14.2 Exercises 5–8

```
\begin{scope}[on background layer]
  \node[region,fit=(A)(B)(D)(E)] {};
\end{scope}
\coordinate (M) at ($(A)!0.5!(B)$);
```

Then revise a label and the shared styles. Repair the architecture only if a dependent object fails to follow.

28.15 ChatGPT Workshop

Ask ChatGPT to review architecture separately from appearance.

1. Extract a structural inventory from this diagram before refactoring it.
2. Propose semantic styles without changing geometry or notation.
3. Separate uniform repetitions from genuine exceptions.
4. Find backgrounds, labels, and bends that should follow named objects.
5. Design revision tests for label length, spacing, and global style changes.

28.16 Final Checklist

Before continuing to Chapter 29, check that you can

1. inventory mathematical and visual roles;
2. give each source section one responsibility;
3. order commands by logical dependency;
4. test a skeleton before adding decoration;
5. distinguish repetition from exception;
6. make annotations and regions follow objects; and
7. test the architecture by revising it.